

DSA & Advanced DSA Roadmap

Phase 7 — Searching Algorithms

Learning Objectives

By the end of this phase, learners will be able to:

- Implement linear search and binary search from scratch, and state the precondition binary search requires (a sorted, or monotonic, search space).
- Explain search space reduction: why halving the candidate range each step yields $O(\log n)$ time.
- Apply binary search beyond plain 'find the target' lookups: locating boundaries (lower/upper bound) and searching over a space of possible answers rather than array indices.
- Adapt binary search to rotated sorted arrays by reasoning about which half remains sorted.
- Solve Easy to Medium level searching problems independently, and approach Hard 'binary search on the answer' problems by identifying the monotonic feasibility check.

Implementation & Internals

Core Concepts

- Linear Search — scanning every element; $O(n)$ but requires no preconditions on the data.
- Binary Search — repeatedly halving a sorted search space by comparing the midpoint to the target.
- Search Space Reduction — the general principle behind binary search: any problem where you can determine 'too high / too low / correct' in $O(1)$ or $O(\log n)$ per check can be searched in $O(\log(\text{range}))$ instead of scanning every candidate.

Implementation Exercises

- Implement binary search iteratively and recursively, and compare their space complexity ($O(1)$ vs $O(\log n)$ due to the call stack).
- Implement a 'lower bound' helper (first index where $\text{nums}[i] \geq \text{target}$) and an 'upper bound' helper (first index where $\text{nums}[i] > \text{target}$) as reusable building blocks.
- Deliberately introduce an off-by-one bug into your binary search (e.g. using $\leq$ vs $<$ incorrectly) and trace through a small example to see it infinite-loop or miss the target, to build intuition for the common pitfalls.

Pattern Mastery

Pattern 1: Lower Bound

Concepts

- Finding the first index where a sorted array's value is greater than or equal to a target.
- Using lower bound as a building block for 'insert in sorted position' and 'first occurrence' queries.

Practice Problems

- Given a sorted array and a target, return the index if found, or the index where it would be inserted to keep the array sorted.

LeetCode

- [Search Insert Position — leetcode.com/problems/search-insert-position](https://leetcode.com/problems/search-insert-position)

Pattern 2: Upper Bound

Concepts

- Finding the first index where a sorted array's value is strictly greater than a target.
- Combining lower bound and upper bound to find the first and last occurrence of a target value in a sorted array with duplicates.

Practice Problems

- Given a sorted array, find the first and last position of a given target value, or $[-1, -1]$ if it doesn't appear.
- Implement binary search itself as a standalone exercise, confirming $O(\log n)$ behavior on a large sorted array.

LeetCode

- [Find First and Last Position of Element in Sorted Array — leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array](https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array)
- [Binary Search — leetcode.com/problems/binary-search](https://leetcode.com/problems/binary-search)

Pattern 3: Binary Search on Answer

Concepts

- Recognizing problems where the question 'what is the minimum/maximum valid X?' has a monotonic feasibility check: once X is large enough (or small enough) to work, every larger (or smaller) X also works.
- Binary searching over the range of possible answers, not over array indices, using a feasibility function as the comparison.

Practice Problems

- Find the minimum eating speed needed to finish all bananas within h hours.
- Find the minimum ship capacity needed to ship all packages within a given number of days.

LeetCode

- [Koko Eating Bananas — leetcode.com/problems/koko-eating-bananas](https://leetcode.com/problems/koko-eating-bananas)
- [Capacity to Ship Packages Within D Days — leetcode.com/problems/capacity-to-ship-packages-within-d-days](https://leetcode.com/problems/capacity-to-ship-packages-within-d-days)

Pattern 4: Rotated Arrays

Concepts

- Recognizing that a rotated sorted array still has one half that is properly sorted at every step, even though the whole array is not.
- Deciding which half to search next by comparing the midpoint against both endpoints, rather than assuming the array is fully sorted.

Practice Problems

- Find the minimum element in a rotated sorted array of unique elements.
- Search for a target value in a rotated sorted array of unique elements.
- Search for a target value in a rotated sorted array that may contain duplicates.

LeetCode

- [Find Minimum in Rotated Sorted Array — leetcode.com/problems/find-minimum-in-rotated-sorted-array](https://leetcode.com/problems/find-minimum-in-rotated-sorted-array)
- [Search in Rotated Sorted Array — leetcode.com/problems/search-in-rotated-sorted-array](https://leetcode.com/problems/search-in-rotated-sorted-array)
- [Search in Rotated Sorted Array II — leetcode.com/problems/search-in-rotated-sorted-array-ii](https://leetcode.com/problems/search-in-rotated-sorted-array-ii)

Phase Assessment

Implementation Tasks

- Implement binary search from scratch, both iteratively and recursively.
- Implement standalone lower-bound and upper-bound helper functions, then use them together to find a value's first and last occurrence.

- Analyze and explain why 'binary search on the answer' is valid only when the feasibility check is monotonic, using Koko Eating Bananas as the example.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/binary-search
- leetcode.com/problems/search-insert-position

Medium Problems

Upper Bound

- leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array

Binary Search on Answer

- leetcode.com/problems/koko-eating-bananas
- leetcode.com/problems/capacity-to-ship-packages-within-d-days

Rotated Arrays

- leetcode.com/problems/find-minimum-in-rotated-sorted-array
- leetcode.com/problems/search-in-rotated-sorted-array
- leetcode.com/problems/search-in-rotated-sorted-array-ii

Hard Problems

- leetcode.com/problems/split-array-largest-sum

On scope and dedup

This phase has 9 distinct problems, larger than the Phase 1–3 baseline. Split Array Largest Sum is included as the Hard-tier binary-search-on-answer problem; it shares its core technique with Capacity to Ship Packages Within D Days (both binary search over a feasibility check) but applies it to a different problem shape (partitioning vs. greedy loading), so it is not a redundant repeat of the Medium problem above it. None of the 9 problems overlaps with any problem used in Phases 0–6.

Coding Challenges (Assessment Day)

- Binary Search
- Find First and Last Position of Element in Sorted Array
- Search in Rotated Sorted Array
- Koko Eating Bananas
- Split Array Largest Sum

Expected Outcome

Students should be able to:

- Implement binary search correctly from scratch, without off-by-one errors, both iteratively and recursively.
- Recognize common searching patterns: lower bound, upper bound, binary search on the answer, and rotated-array search.
- Solve unseen searching problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium searching questions independently, and approach Hard binary-search-on-answer problems by identifying the monotonic feasibility check.